

Laboratory Report: Lab 1 List

Zofia Warzycha, 280971

March 2026

1 Task 1: Box Class

1.1 Task Description

For the first task, I implemented a `Box` class with three instance variables of type `double`: `width`, `height`, and `depth`, all defaulting to 1.0. I created a default constructor and a three-parameter constructor. I also added methods to calculate the total surface area (`getSurfaceArea()`) and the volume (`getVolume()`). Finally, I tested the code using a `BoxTester` class.

1.2 Source Code

Listing 1: Box and BoxTester classes

```
1 public class Box {
2     private double width;
3     private double height;
4     private double depth;
5
6     public Box() {
7         width = 1.0;
8         height = 1.0;
9         depth = 1.0;
10    }
11
12    public Box(double width, double height, double depth) {
13        this.width = width;
14        this.height = height;
15        this.depth = depth;
16    }
17
18    public double getWidth() { return width; }
19    public double getHeight() { return height; }
20    public double getDepth() { return depth; }
21
22    public double getSurfaceArea() {
23        return 2 * (width * height + width * depth + height * depth);
24    }
25    public double getVolume() {
26        return width * height * depth;
27    }
28 }
```

```

29
30 public class BoxTester {
31     public static void main(String[] args) {
32         Box box1 = new Box();
33         Box box2 = new Box(2.0, 3.0, 4.0);
34
35         System.out.println("Box_1:");
36         System.out.println("Width:" + box1.getWidth());
37         System.out.println("Height:" + box1.getHeight());
38         System.out.println("Depth:" + box1.getDepth());
39         System.out.println("Surface_Area:" + box1.getSurfaceArea());
40         System.out.println("Volume:" + box1.getVolume());
41
42         System.out.println();
43
44         System.out.println("Box_2:");
45         System.out.println("Width:" + box2.getWidth());
46         System.out.println("Height:" + box2.getHeight());
47         System.out.println("Depth:" + box2.getDepth());
48         System.out.println("Surface_Area:" + box2.getSurfaceArea());
49         System.out.println("Volume:" + box2.getVolume());
50     }
51 }

```

1.3 Console Output

```

Box 1:
Width: 1.0
Height: 1.0
Depth: 1.0
Surface Area: 6.0
Volume: 1.0

Box 2:
Width: 2.0
Height: 3.0
Depth: 4.0
Surface Area: 52.0
Volume: 24.0

```

2 Task 2: PrepaidCard Class

2.1 Task Description

In the second task, I wrote a `PrepaidCard` class to model a card with an integer `balance` variable (default value 100). The class has methods to increase the balance (`topUp()`) and decrease it (`pay()`). I added an if-statement to print "Insufficient funds" if the balance is too low. For extra points, I also implemented the `transferTo()` method to move funds from one card to another.

2.2 Source Code

Listing 2: PrepaidCard and PrepaidCardTester classes

```
1 public class PrepaidCard {
2     private int balance;
3
4     public PrepaidCard() {
5         balance = 100;
6     }
7
8     public PrepaidCard(int balance) {
9         this.balance = balance;
10    }
11
12    public int getBalance() { return balance; }
13
14    public void topUp(int amount) {
15        this.balance += amount;
16    }
17
18    public void pay(int amount) {
19        if (balance < amount) {
20            System.out.println("Insufficient_funds!");
21        } else {
22            this.balance -= amount;
23        }
24    }
25
26    public void transferTo(PrepaidCard anotherCard, int amount) {
27        if (balance < amount) {
28            System.out.println("Insufficient_funds_to_transfer!");
29        } else {
30            this.balance -= amount;
31            anotherCard.topUp(amount);
32        }
33    }
34 }
35
36 public class PrepaidCardTester {
37     public static void main(String[] args) {
38         PrepaidCard Card1 = new PrepaidCard();
39         PrepaidCard Card2 = new PrepaidCard();
40
41         System.out.println("Starting_balances:");
42         System.out.println("Card_1:_ " + Card1.getBalance());
```

```

43         System.out.println("Card2: " + Card2.getBalance());
44
45         System.out.println("\nTopping up Card1 with 100:");
46         Card1.topUp(100);
47         System.out.println("Card1 is now: " + Card1.getBalance());
48
49         System.out.println("\nTopping up Card2 with 50:");
50         Card2.topUp(50);
51         System.out.println("Card1 is now: " + Card1.getBalance());
52         System.out.println("Card2 is now: " + Card2.getBalance());
53
54         System.out.println("\nPaying with Card2 for a 10$ pizza slice:");
55         Card2.pay(10);
56         System.out.println("Card1 is now: " + Card1.getBalance());
57         System.out.println("Card2 is now: " + Card2.getBalance());
58
59         System.out.println("\nTransferring 50$ from Card1 to Card2:");
60         Card1.transferTo(Card2, 50);
61         System.out.println("Card1 is now: " + Card1.getBalance());
62         System.out.println("Card2 is now: " + Card2.getBalance());
63     }
64 }

```

2.3 Console Output

Starting balances:

Card 1: 100

Card 2: 100

Topping up Card1 with 100:

Card 1 is now: 200

Topping up Card2 with 50:

Card 1 is now: 200

Card 2 is now: 150

Paying with Card2 for a 10\$ pizza slice:

Card 1 is now: 200

Card 2 is now: 140

Transferring 50\$ from Card1 to Card2:

Card 1 is now: 150

Card 2 is now: 190

3 Task 3: Space Station Management

3.1 Task Description

The final task was a larger program to manage a space station. I completed three parts:

1. **Commander:** I made a `CrewMember` profile that holds the name, age, role, specialization, and mission hours. I used a Java `enum` for the roles.
2. **Quartermaster:** I added a method to calculate daily oxygen. The default is 2 units, but it goes up to 3 if they have over 1000 mission hours. It also divides the oxygen based on how many distributions happen per day.
3. **Station Tracker:** I created an array for 10 crew members. Then, I wrote a loop to print all the profiles and calculate the average age and average mission hours.

3.2 Crucial Parts of the Source Code

Here are the most important parts of the code showing the logic for the profile, oxygen distribution, and the array loop.

Listing 3: Core logic within the `CrewMember` class

```
1 public enum Role {
2     COMMANDER, PILOT, ENGINEER, SCIENTIST, UNKNOWN
3 }
4
5 public class CrewMember {
6     // ... instance variables and constructors ...
7
8     public void printProfile() {
9         System.out.println("Hello! Meet " + this.name + ", a " + this.
10             age + "-year-old" +
11             this.role + ". This crew member specializes in " + this.
12                 .specialization +
13                 " and has already completed " + this.missionHours + "
14                 mission hours.");
15     }
16
17     public void distributeOxygen(int dailyDistributions) {
18         int dailyOxygen;
19         if (this.missionHours > 1000) {
20             dailyOxygen = 3;
21         } else {
22             dailyOxygen = 2;
23         }
24         double perDistribution = (double) dailyOxygen /
25             dailyDistributions;
26         System.out.println(this.name + " needs " + dailyOxygen +
27             " oxygen units daily, which means " + perDistribution +
28             " oxygen units per distribution.");
29     }
30 }
```

Listing 4: Safe array loop inside `StationTracker`

```
1 public class StationTracker {
```

```

2   public static void main(String[] args) {
3       CrewMember[] stationCrew = new CrewMember[10];
4
5       // ... adding crew members ...
6
7       int totalAge = 0;
8       double totalHours = 0.0;
9       int activeStationCrewCount = 0;
10
11      for (int i = 0; i < stationCrew.length; i++) {
12          if (stationCrew[i] != null) {
13              stationCrew[i].printProfile();
14              totalAge += stationCrew[i].getAge();
15              totalHours += stationCrew[i].getMissionHours();
16              activeStationCrewCount++;
17          }
18      }
19
20      double averageAge = (double) totalAge / activeStationCrewCount;
21      double averageHours = totalHours / activeStationCrewCount;
22  }
23 }

```

3.3 Console Output

Hello! Meet Unknown, a 0-year-old UNKNOWN. This crew member specializes in Unknown and has already completed 0.0 mission hours.

Hello! Meet Zofia Warzycha, a 21-year-old ENGINEER. This crew member specializes in IT and has already completed 29.6 mission hours.

Julia Sadowska needs 2 oxygen units daily, which means 0.5 oxygen units per distribution.

Maja Kaczmarek needs 3 oxygen units daily, which means 1.5 oxygen units per distribution.

Station crew profiles:

Hello! Meet Igor Stelmach, a 21-year-old SCIENTIST. This crew member specializes in Astrophysics and has already completed 747.0 mission hours.

Hello! Meet Hanna Szpak, a 20-year-old ENGINEER. This crew member specializes in Nuclear physics and has already completed 668.0 mission hours.

Hello! Meet Sebastian Kucharski, a 22-year-old COMMANDER. This crew member specializes in Organizational management and has already completed 9965.0 mission hours.

Average age of the crew: 21.0

Average hours of the crew: 3793.3333333333335

4 Conclusions

- I learned how to use Object-Oriented Programming to store data inside objects using private fields and how to change that data using methods.

- Using different constructors makes it easy to create objects either with default values or specific data.
- I learned how to store objects inside an array. Checking if `stationCrew[i] != null` was very important so the program wouldn't crash when looping through empty spots in the array.